🚀 Delivery Manager v2.0
Professional Delivery Management System

[![Python](https://img.shields.io/badge/Python-3.11+-blue.svg)](https://www.python.org/)
[![Flask](https://img.shields.io/badge/Flask-2.3.3-lightgrey.svg)](https://flask.palletsprojects.com/)
[![SQLite](https://img.shields.io/badge/Database-SQLite-003B57.svg)](https://www.sqlite.org/)
[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT)

Delivery Manager is a production-ready, full-stack management system designed for couriers and delivery companies. It features a robust multi-role architecture, dynamic pricing engine, and responsive mobile-first design.

🌟 Key Features

👤 Client / User
* **Dynamic Order Creation:** Real-time price calculation based on distance and category.
* **Live Tracking:** Monitor order status from "New" to "Delivered".
* **Feedback System:** Rate couriers and leave detailed reviews.
* **Order History:** Complete log of all past deliveries.

🚴 Courier
* **Order Browser:** View available orders nearby with distance calculations.
* **Status Management:** Seamlessly update delivery progress.
* **Earnings Tracker:** Monitor ratings and performance metrics.
* **Mobile-First UI:** Optimized for touch-screen devices in the field.

👨‍💼 Administrator
* **Global Dashboard:** Real-time system statistics and revenue monitoring.
* **User Management:** Control roles (User, Courier, Admin) and permissions.
* **Audit Trail:** Complete activity logging for security and oversight.
* **Database Management:** Full visibility into orders, users, and reviews.

🛠️ Technology Stack

Layer	Technologies

Backend	Python 3.11, Flask, Flask-JWT-Extended, Flask-CORS
Frontend	HTML5, CSS3 (Modern Flexbox/Grid), Vanilla JavaScript (ES6+)
Database	SQLite 3
Security	JWT (JSON Web Tokens), Password Hashing (Werkzeug)
Deployment	Docker, Docker Compose, Nginx
PWA	Service Workers, IndexedDB, Manifest.json

📐 System Architecture

The application follows a clean **Client-Server** architecture:
* **Frontend (Port 8000):** A responsive web application served via Python's HTTP server or Nginx.
* **Backend (Port 5000):** A RESTful API built with Flask, providing secure endpoints for all operations.
* **Auth Layer:** Secure JWT-based authentication with role-based access control.

🚦 Getting Started

Prerequisites
* Python 3.11 or higher
* pip (Python package manager)

Quick Setup & Launch
The easiest way to get the system running is using the all-in-one launcher:

1. **Install dependencies:**
   ```bash
   pip install -r backend/requirements.txt
   ```

2. **Run the application:**
   ```bash
   python run_all.py
   ```

3. **Access the Application:**
   * **Frontend:** http://localhost:8000
   * **Backend API:** http://localhost:5000

🔐 Default Credentials

Role	Username	Password
Admin	`admin`	`admin123`

💰 Business Model & Pricing
The system uses a dynamic pricing formula:
Price = Base ($15) × Distance × Category Multiplier × Urgency Multiplier

* **Categories:** Beverages (1.0x), Food (1.5x), Electronics (2.0x), Documents (0.8x).
* **Revenue Split:** 80% to Courier, 20% Platform Commission.

📂 Project Structure
```bash
.
├── backend/          # Flask API & Database logic
├── frontend/         # PWA, HTML, CSS, and JS logic
├── docs/             # Project documentation (PDF/MD)
├── docker-compose.yml  # Container orchestration
└── run_all.py        # Unified system launcher
```

👥 Development Team (Group 10)
* **Oleksandr Nechyporenko** - Project Leader
* **Stanislaw Szary** - System Architect
* **Vladyslav Khanchych** - Lead Developer & Tester

Created for the "Projekt Zespołowy" course, Semester Summer 2025/2026. Directed by Paweł Sikora.